EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from
© Pearson Education
Hengzhao Yang
Zhifeng Zhu
Yajun Ha

上海科技大学
ShanghaiTech University

Sequential Logic Circuit

Finite State Machine

Part1:
Concepts
Programming

State Machine

Sequential Circuit

Output depends on past behavior and present input.

Finite State Machine

A state machine is a sequential circuit having a limited (finite) number of states occuring in a prescribed order.

State Machine

Example: Elevator Control System

States: Idle, Moving Up, Moving Down, Door Open

Transitions:

Idle → Moving Up (if call button pressed above)

Idle → Moving Down (if call button pressed below)

Moving Up/Down → Door Open (when reaching a floor)

Door Open → Idle (after a timeout)

State Machine

Two basic types of state machines are the

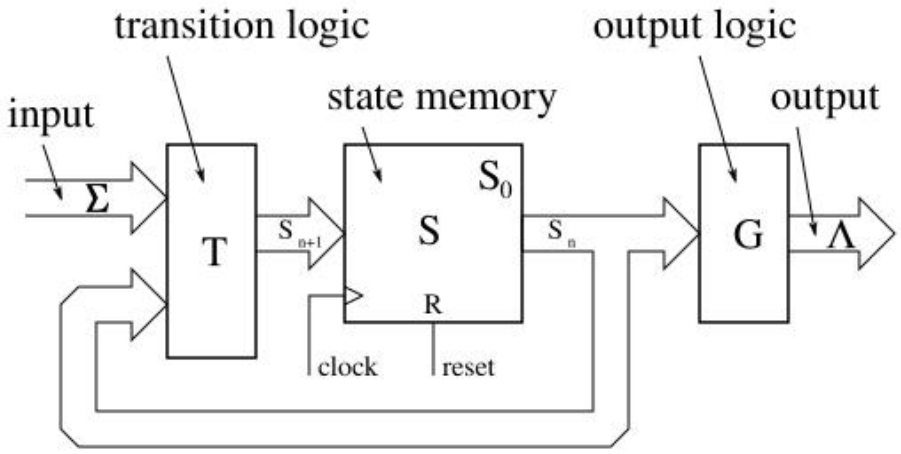
- Moore state machine:
  - Outputs depend only on the internal present state
- Mealy state machine
  - Outputs depend on both the internal present state and on the inputs

Moore Machine

Output depends on internal states, not directly on inputs.

➡ Flip-flops store the present state.

➡ Combinational logic reads the present state and computes the next state.

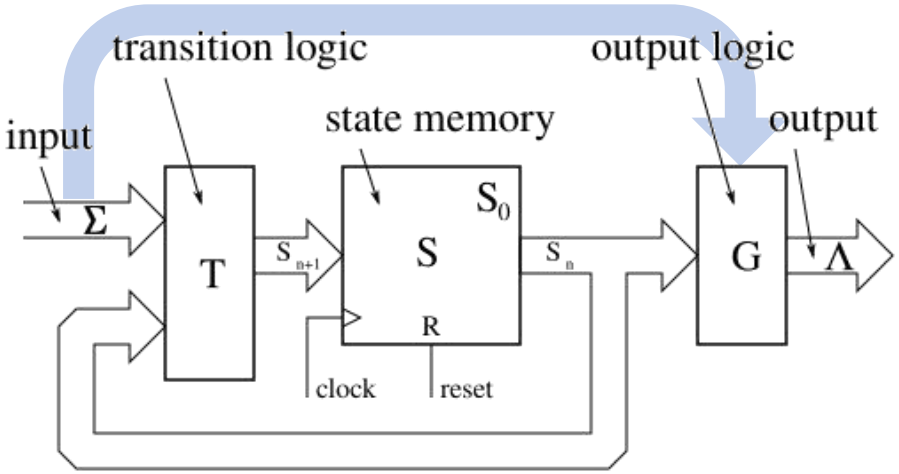


Outputs depend only on the internal state

https://en.wikipedia.org/wiki/Moore_machine

Mealy Machine

The output of depends on both the present state and inputs, making this a mealy machine.



Outputs depend on the internal state, and inputs

Design flow

Example: Sequence detector

Design specifications (problem statement)

- Circuit has one input (w) and one output (z)
- All changes occur at positive clock edges
- Operation: z=1 if w=1 for continued two clock cycles; z=0, otherwise

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

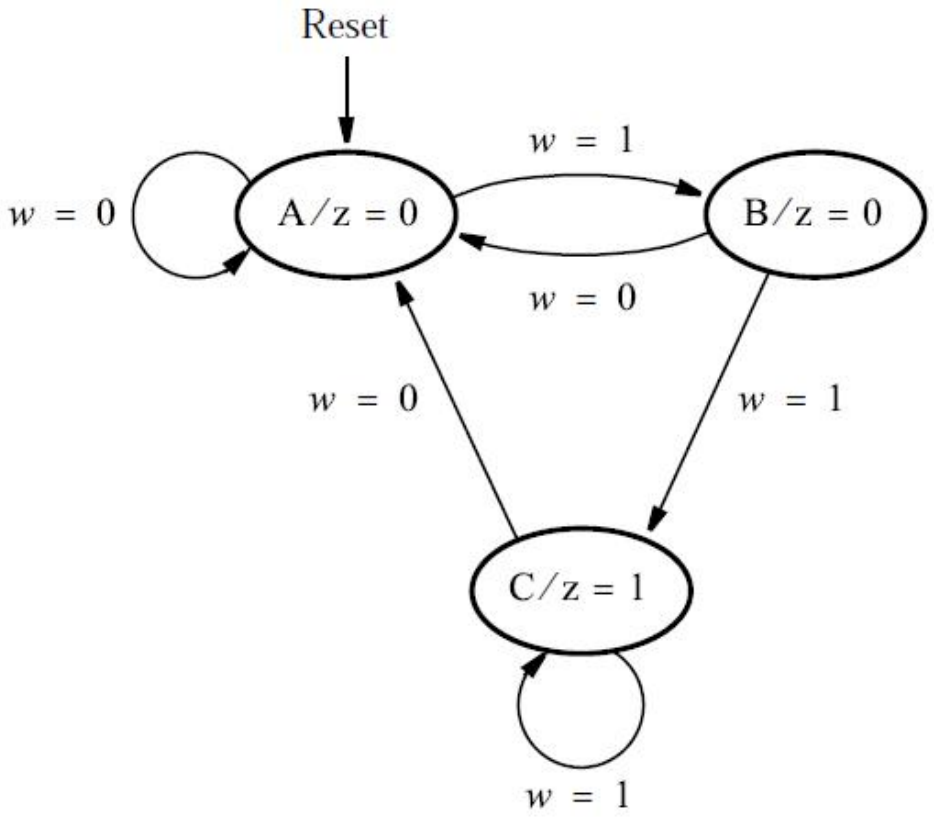
State diagram

Identify states to describe circuit behavior

- State A (starting state): when a reset signal is applied or w=0, circuit enters this state, z=0
- State B: w=1 for one clock cycle, z=0
- State C: w=1 for two clock cycles, z=1

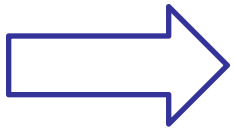
State diagram

- Nodes: states
- Directed arcs: state transitions
- State A (starting state): when a reset signal is applied or w=0, circuit enters this state, z=0
- State B: w=1 for one clock cycle, z=0
- State C: w=1 for two clock cycles, z=1



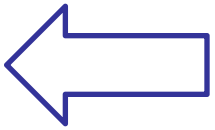
Code 1 for State-machine (VHDL)

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY simple IS
    PORT ( Clock, Resetn, w    : IN    STD_LOGIC ;
           z                   : OUT   STD_LOGIC ) ;
END simple ;

ARCHITECTURE Behavior OF simple IS
    TYPE State_type IS (A, B, C) ;
    SIGNAL y_present, y_next : State_type ;
```



Determines next state

```
BEGIN
    PROCESS ( w, y_present )
    BEGIN
        CASE y_present IS
            WHEN A =>
                IF w = '0' THEN
                    y_next <= A ;
                ELSE
                    y_next <= B ;
                END IF ;
            WHEN B =>
                IF w = '0' THEN
                    y_next <= A ;
                ELSE
                    y_next <= C ;
                END IF ;
            WHEN C =>
                IF w = '0' THEN
                    y_next <= A ;
                ELSE
                    y_next <= C ;
                END IF ;
        END CASE ;
    END PROCESS ;
```



Determines output

```
    PROCESS (Clock, Resetn)
    BEGIN
        IF Resetn = '0' THEN
            y_present <= A ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            y_present <= y_next ;
        END IF ;
    END PROCESS ;

    z <= '1' WHEN y_present = C ELSE '0' ;
END Behavior ;
```

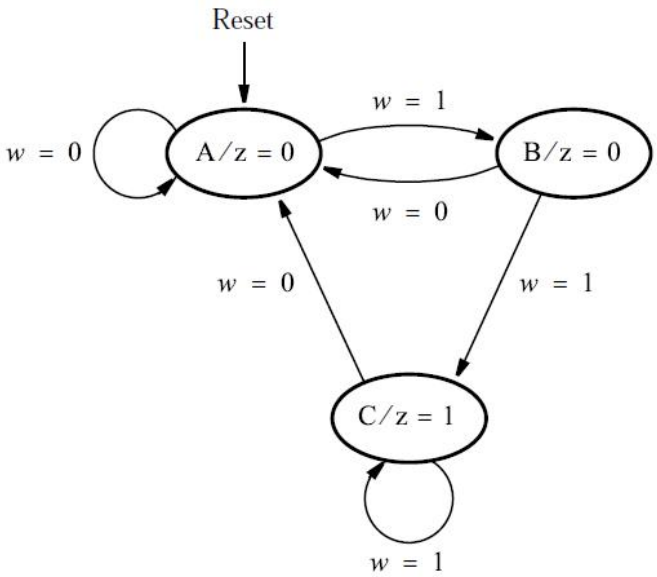
Code 2 for State-machine (VHDL)

Improve 1 by using
When ... Others

```
    PROCESS ( Clock, Resetn )
    BEGIN
        IF Resetn = '0' THEN
            y_present <= A ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            y_present <= y_next ;
        END IF ;
    END PROCESS ;
    z <= '1' WHEN y_present = C ELSE '0' ;
END Behavior ;
```

```
BEGIN
    PROCESS ( w, y_present )
    BEGIN
        CASE y_present IS
            WHEN A =>
                IF w = '0' THEN y_next <= A ;
                ELSE y_next <= B ;
                END IF ;
            WHEN B =>
                IF w = '0' THEN y_next <= A ;
                ELSE y_next <= C ;
                END IF ;
            WHEN C =>
                IF w = '0' THEN y_next <= A ;
                ELSE y_next <= C ;
                END IF ;
            WHEN OTHERS =>
                y_next <= A ;
        END CASE ;
    END PROCESS ;
```

Code for State-machine (Verilog)




```
1   module fsm (
2       input clk, reset, w,
3       output reg z
4   );
5
6       reg [1:0] state;
7
8       parameter A = 2'b00, B = 2'b01, C = 2'b10;
9
10      always @(posedge clk or posedge reset) begin
11          if (reset)
12              state <= A;
13          else begin
14              case (state)
15                  A: state <= w ? B : A;
16                  B: state <= w ? C : A;
17                  C: state <= w ? C : A;
18                  default: state <= A;
19              endcase
20          end
21      end
22
23      always @(*) begin
24          case (state)
25              C: z = 1'b1;
26              default: z = 1'b0;
27          endcase
28      end
29
30  endmodule
```

State transition

Output control

Course Project (2026)

How to design state-machine?

How to test your code?

STATE:
IDLE
VALID
PASS
CLOSE
ALARM
...

Project: Subway Turnstile Controller

Project Overview

In 2025, the Shanghai Metro transported 3.71 billion passengers, ranking first nationwide, with an average daily ridership of 10.15 million. Every turnstile in every station silently completes the cycle of "tap card → verify → allow passage → close." Behind this seemingly simple action lies a precise digital logic control system — one that must respond to sensor signals in real time, determine the direction of travel, handle unauthorized entry, and ensure passenger safety under strict timing constraints.

This project requires you to design a **subway turnstile controller** digital logic circuit. Starting from a basic two-state turnstile, you will progressively add multi-state access control, direction detection, and more, ultimately building a complete control system that closely resembles real turnstile behavior. The project is divided into 4 parts, each with **independent module files and testbenches**, where each subsequent part incrementally adds functionality.

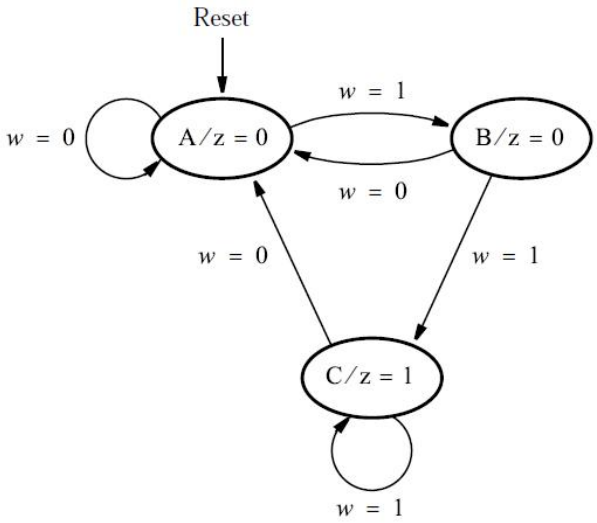
Finite State Machine

Part 2:

Design a State Machine Circuit

State table

- State table is a more structured way of representing states.
- Equivalent to state diagram.



Node	Arrow		Output
Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

Step 1: Assign variables to states

- Each state is represented by a combination of state variables
- Three states (A, B, C) need two state variables (y_1, y_2)
- Moore type: output only depends on state
- Present-state variables: y_1, y_2
- Next-state variables: Y_1, Y_2
- A: 00, B: 01, C: 10. Note: “11” is not used

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

	Present state	Next state		Output
		$w = 0$	$w = 1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

上海科技大学
ShanghaiTech University

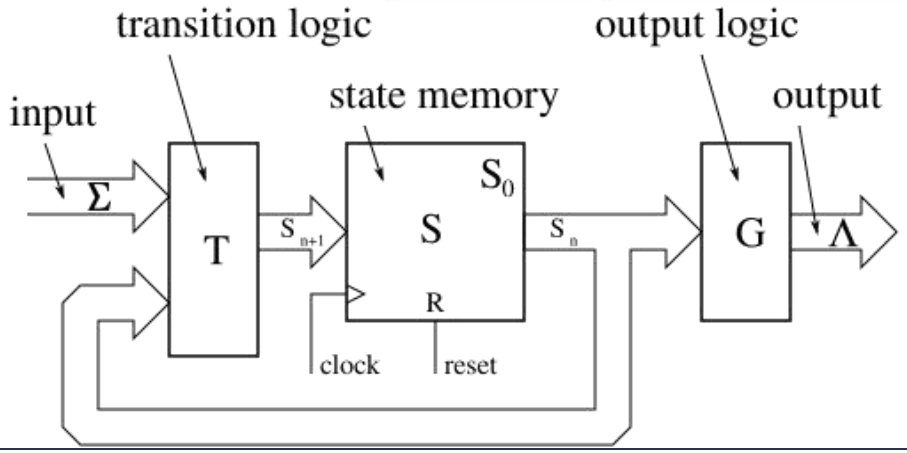
Step 2: Design a state machine

Given present state:

Determine:

- next state Y1,
- next state Y2,
- output

	Present state	Next state		Output
		$w = 0$	$w = 1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

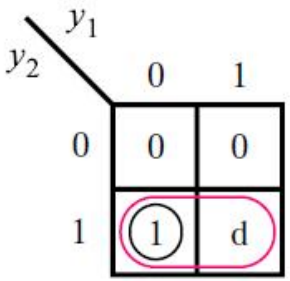


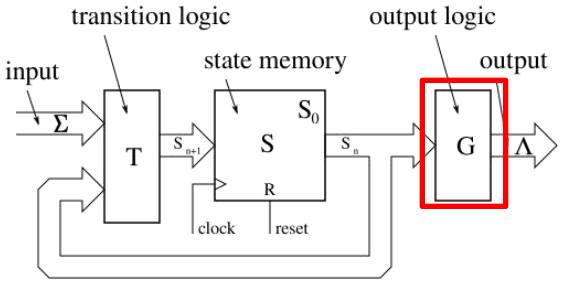
Step 2: Output expression

Output expression: z

- Ignoring don't cares: $z = \bar{y}_1 y_2$
- Using don't cares: $z = y_2$

y2	y1	z
0	0	0
0	1	0
1	0	1
1	1	d





	Present state $y_2 y_1$	Next state $w=0$ $Y_2 Y_1$	Next state $w=1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

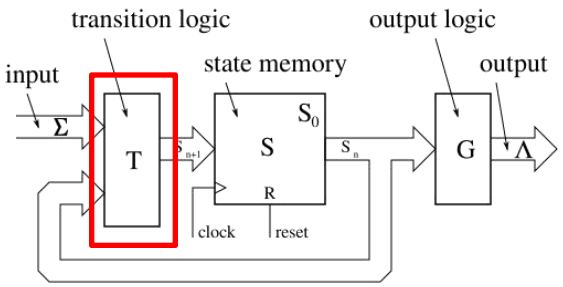
Step 2: Next-state expressions

Next-state expression: Y_1

- Ignoring don't cares: $Y_1 = w\bar{y}_1\bar{y}_2$
- Using don't cares: $Y_1 = w\bar{y}_1\bar{y}_2$

w	y2	y1	Y_1
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	d
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	d

w \ y_2y_1	00	01	11	10
0	0	0	d	0
1	1	0	d	0



	Present state	Next state		Output
		$w = 0$	$w = 1$	
	y_2y_1	Y_2Y_1	Y_2Y_1	z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

Step 2: Next-state expressions

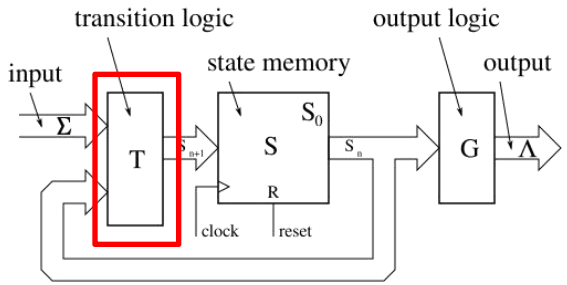
Next-state expression: Y_2

- Ignoring don't cares: $Y_2 = wy_1\bar{y}_2 + w\bar{y}_1y_2$
- Using don't cares:

$$Y_2 = wy_1 + wy_2 = w(y_1 + y_2)$$

w	y2	y1	Y_2
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	d
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	d

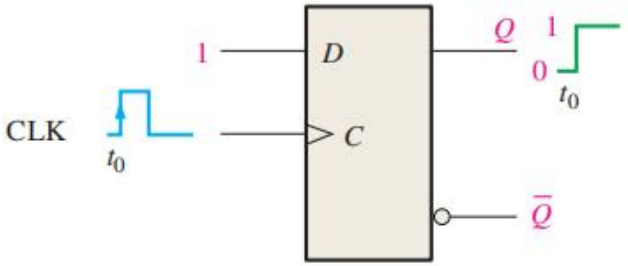
w \ y_2y_1	00	01	11	10
0	0	0	d	0
1	0	1	d	1



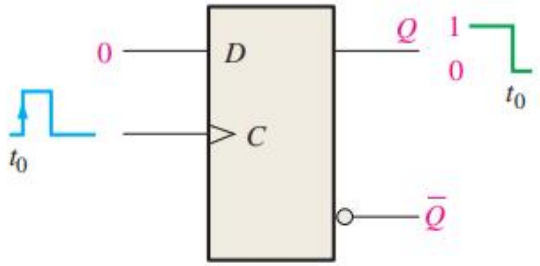
	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	y_2y_1	Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Step 2: D Flip flop

State machine needs storage for states.
A D Flip-Flop is a sequential logic element that stores a single bit of data.



(a) $D = 1$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)



(b) $D = 0$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)

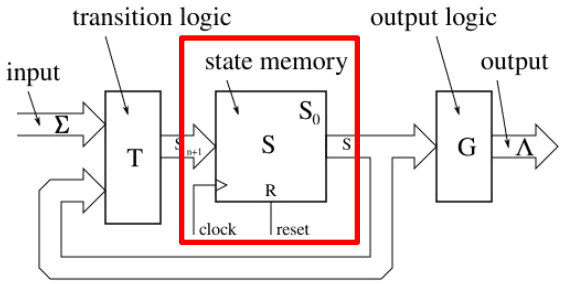


TABLE 7–2

Truth table for a positive edge-triggered D flip-flop.

Inputs		Outputs		
D	CLK	Q	$\overline{Q}$	Comments
0	↑	0	1	RESET
1	↑	1	0	SET

↑ = clock transition LOW to HIGH

Step 3: Draw Circuit (using don't cares)

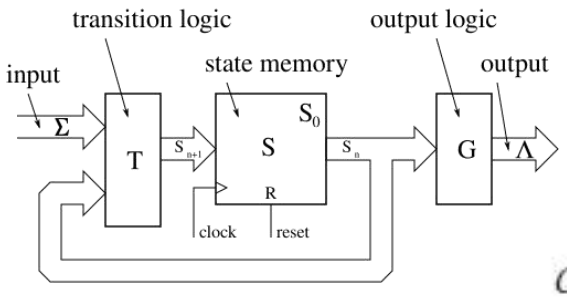
Output

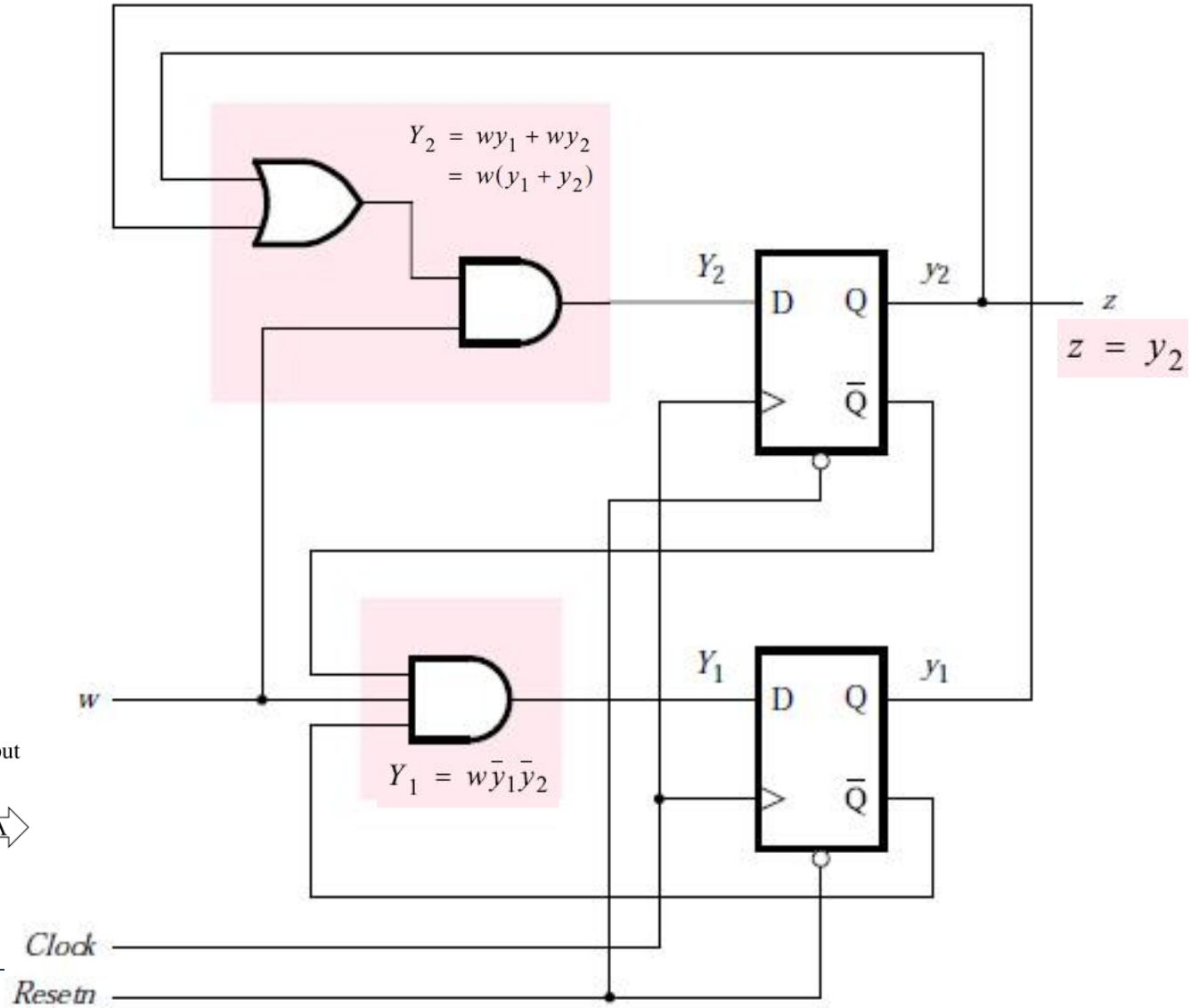
$$z = y_2$$

Next-state

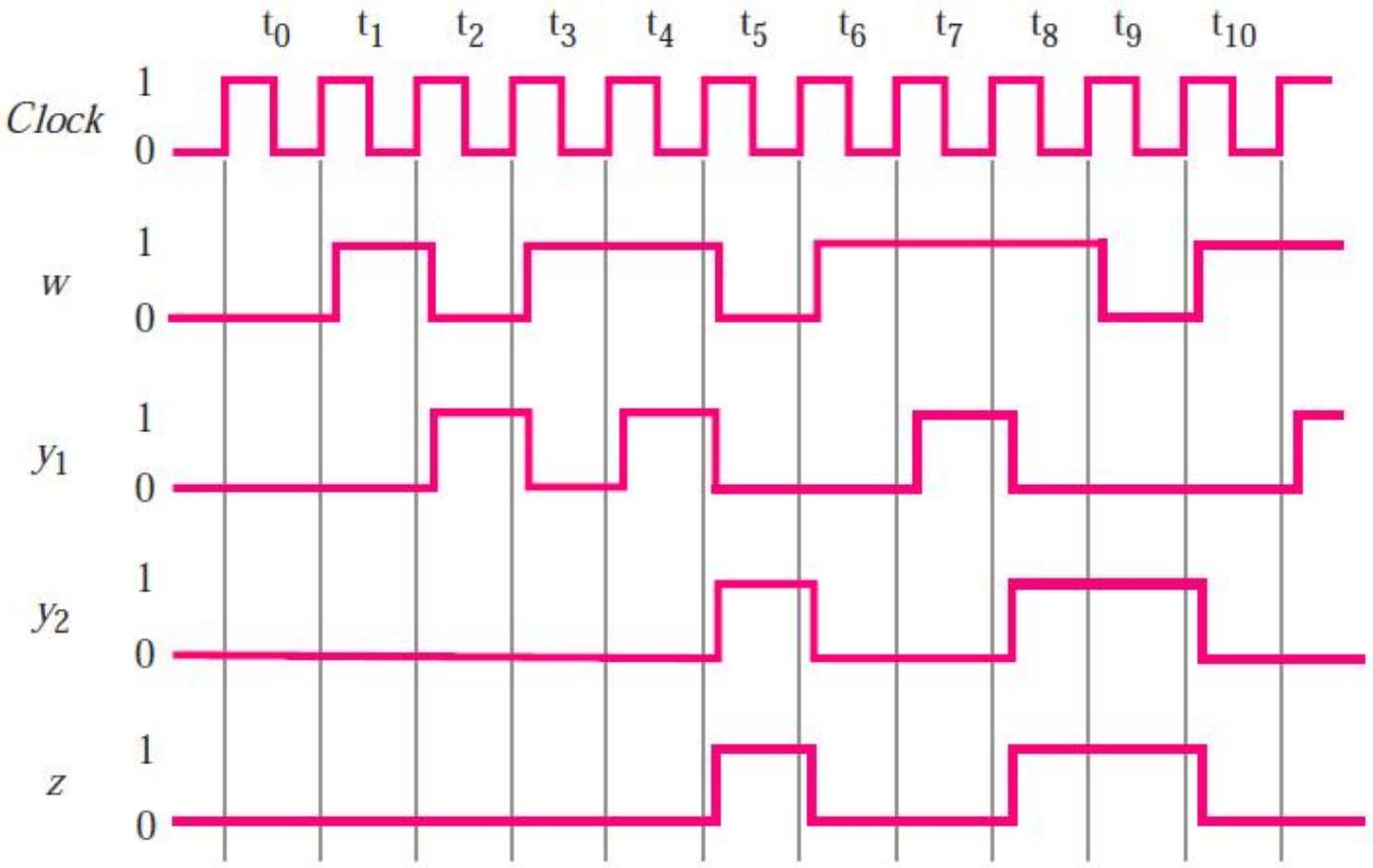
$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$Y_2 = wy_1 + wy_2 = w(y_1 + y_2)$$





Timing diagram



Other design choices

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

Original state assignment

- A: 00, B: 01, C: 10
- "11" is not used

	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Alternative state assignment

- A: 00, B: 01, C: 11
- "10" is not used

	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

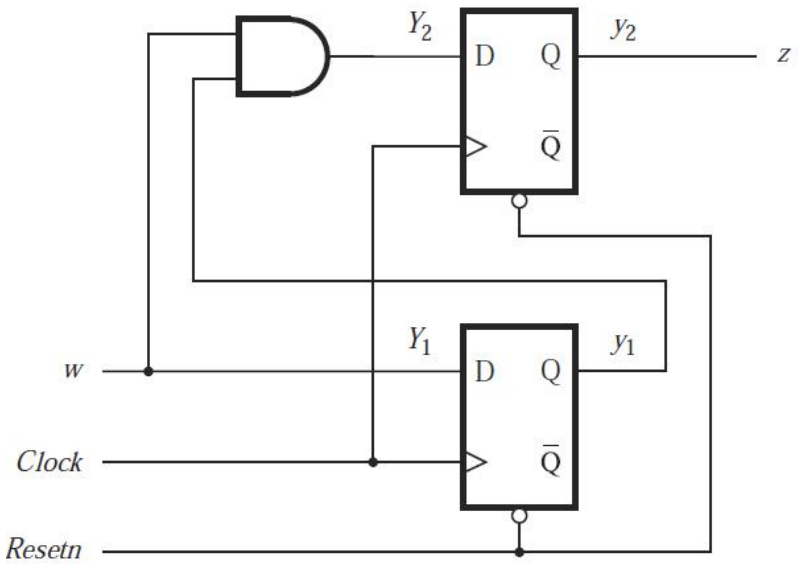
Other design choices

Expressions

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

$$z = y_2$$



	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	*dd*	*dd*	*d*

One-hot encoding

One-hot encoding

- Another approach to assign states
- Number of states equals number of state variables
- For each state, only ONE state variable is “hot”: its value is “1”. The values of all the other state variables are “0”.

Example

State table

- Three states: A, B, C

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

One-hot state assignment

- Three state variables: y_3, y_2, y_1
- A: 001, B: 010, C: 100
- Other combinations of state variables are don't cares

	Present state	Next state		Output
		$w = 0$	$w = 1$	z
	$y_3 y_2 y_1$	$Y_3 Y_2 Y_1$	$Y_3 Y_2 Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Example

- Output and next-state expressions

$$Y_1 = \overline{w}$$
$$Y_2 = wy_1$$
$$Y_3 = w\overline{y}_1$$
$$z = y_3$$

	Present state	Next state		Output
		$w = 0$	$w = 1$	
	$y_3 y_2 y_1$	$Y_3 Y_2 Y_1$	$Y_3 Y_2 Y_1$	z
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

- Circuit is not simpler for this example
- One-hot encoding is useful in many cases